OPERATING SYSTEM

PRACTICAL – EXERCISE

1. Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.

```
#include <stdio.h>
#include <windows.h>

int main() {
 STARTUPINFOA si = {0};
 PROCESS_INFORMATION pi = {0};

 si.cb = sizeof(si);

 printf("Parent Process\n");
 printf("Process ID (PID): %lu\n", GetCurrentProcessId());

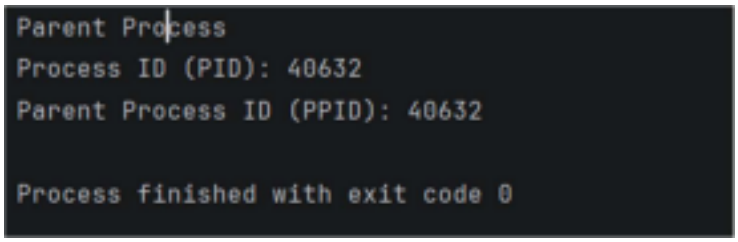
 if (CreateProcessA(
 "C:\\Windows\\System32\\notepad.exe",
 NULL, NULL, NULL, FALSE, 0,
 NULL, NULL, &si, &pi)) {


 printf("Parent Process ID (PPID): %lu\n",
 GetCurrentProcessId());

 CloseHandle(pi.hProcess);
 CloseHandle(pi.hThread);
 } else {
 printf("Process creation failed: %lu\n", GetLastError());
 }

 return 0;
}
```

OUTPUT



2. Create a new process by invoking the appropriate system call. Get the process identifier of the

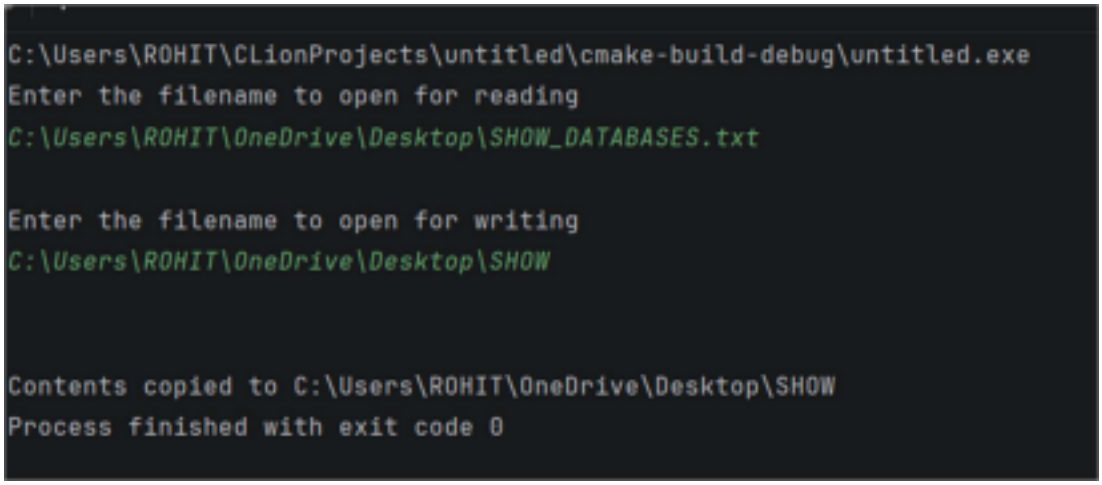
currently running process and its respective parent using system calls and display the same

using a C program.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
 FILE *fptr1, *fptr2;
 char filename[100], c;
 printf("Enter the filename to open for reading \n");
 scanf("%s", filename);
 fptr1 = fopen(filename, "r");
 if (fptr1 == NULL)
 {
 printf("Cannot open file %s \n", filename);
 exit(0);
 }
 printf("Enter the filename to open for writing \n");
 scanf("%s", filename);
 fptr2 = fopen(filename, "w");
 if(fptr2 == NULL)
 {
 printf("Cannot open file %s \n", filename);
 exit(0);
 }
```

```
 c = fgetc(fptr1);
 while (c != EOF)
 {
 fputc(c, fptr2);
 c = fgetc(fptr1);
 }
 printf("\nContents copied to %s", filename);
 fclose(fptr1);
 fclose(fptr2);
 return 0;
}
```

OUTPUT



3. Design a CPU scheduling program with C using First Come First Served technique with the

following considerations.

a. All processes are activated at time 0.

b. Assume that no process waits on I/O devices.

```
#include <stdio.h>
```

```c
int main()
{
   int n, i;
   int bt[20], wt[20], tat[20];
   float avg_wt = 0, avg_tat = 0;

   printf("Enter the number of processes: ");
   scanf("%d", &n);

   printf("Enter burst time for each process:\n");
   for (i = 0; i < n; i++)
   {
       printf("P%d: ", i + 1);
       scanf("%d", &bt[i]);
   }

   wt[0] = 0;

   for (i = 1; i < n; i++)
   {
       wt[i] = wt[i - 1] + bt[i - 1];
   }

   for (i = 0; i < n; i++)
   {
       tat[i] = wt[i] + bt[i];
       avg_wt += wt[i];
       avg_tat += tat[i];
```

```
    }

    avg_wt /= n;
    avg_tat /= n;

    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

    for (i = 0; i < n; i++)
    {
        printf("P%d\t%d\t\t%d\t\t%d\n",
                i + 1, bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

```
C:\Users\ROHIT\CLionProjects\untitled\cmake-build-debug\untitled.exe
Enter the number of processes:5

Enter burst time for each process:
P1:2

P2:1

P3:4

P4:7

P5:5


Process Burst Time      Waiting Time    Turnaround Time
P1      2               0               2
P2      1               2               3
P3      4               3               7
P4      7               7               14
P5      5               14              19

Average Waiting Time = 5.20
Average Turnaround Time = 9.00

Process finished with exit code 0
```

4.

```c
#include <stdio.h>


int main()

{

   int n, i, j, temp;

   int bt[20], p[20], wt[20], tat[20];

   float avg_wt = 0, avg_tat = 0;


   printf("Enter the number of processes: ");

   scanf("%d", &n);


   printf("Enter burst time for each process:\n");

   for (i = 0; i < n; i++)

   {
```

```
    p[i] = i + 1;
    printf("P%d: ", p[i]);
    scanf("%d", &bt[i]);
}

// Sort processes by burst time
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (bt[i] > bt[j])
        {
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;

            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

wt[0] = 0;

for (i = 1; i < n; i++)
    wt[i] = wt[i - 1] + bt[i - 1];
```

```
    for (i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    avg_wt /= n;
    avg_tat /= n;

    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

    for (i = 0; i < n; i++)
        printf("P%d\t%d\t\t%d\t\t%d\n",
               p[i], bt[i], wt[i], tat[i]);

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

```
C:\Users\ROHIT\CLionProjects\untitled\cmake-build-debug\untitled.exe
Enter the number of processes:3

Enter burst time for each process:
P1:1

P2:1

P3:1


Process Burst Time      Waiting Time    Turnaround Time
P1      1               0               1
P2      1               1               2
P3      1               2               3

Average Waiting Time = 1.00
Average Turnaround Time = 2.00

Process finished with exit code 0
```

5. Construct a scheduling program with C that selects the waiting process with the highest

priority to execute next.

```c
#include <stdio.h>


int main()

{

   int n, i, j, temp;

   int bt[20], p[20], pr[20], wt[20], tat[20];

   float avg_wt = 0, avg_tat = 0;
```

```c
printf("Enter the number of processes: ");
scanf("%d", &n);

printf("Enter burst time and priority for each process:\n");

for (i = 0; i < n; i++)
{
    p[i] = i + 1;

    printf("P%d burst time: ", p[i]);
    scanf("%d", &bt[i]);

    printf("P%d priority: ", p[i]);
    scanf("%d", &pr[i]);
}

// Sort by priority (smaller number = higher priority)
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (pr[i] > pr[j])
        {
            temp = pr[i];
            pr[i] = pr[j];
            pr[j] = temp;

            temp = bt[i];
```

```
                bt[i] = bt[j];
                bt[j] = temp;

                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    wt[0] = 0;

    for (i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for (i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    avg_wt /= n;
    avg_tat /= n;

    printf("\nProcess\tPriority\tBurst Time\tWaiting Time\tTurnaround
Time\n");
```

```
    for (i = 0; i < n; i++)
    {
        printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n",
                p[i], pr[i], bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

```
C:\Users\ROHIT\CLionProjects\untitled\cmake-build-debug\untitled.exe
Enter the number of processes:1

Enter burst time and priority for each process:
P1 burst time:2

P1 priority:

3


Process Priority        Burst Time      Waiting Time    Turnaround Time
P1      3               2               0               2

Average Waiting Time = 0.00
Average Turnaround Time = 2.00

Process finished with exit code 0
```